

Práctica 1: Analizadores léxicos con Lex (Flex)

Compiladores 2026-2

Profesora: Ariel Adara Mercado Martínez

Alumno

Edwin Rodríguez Nevarez - 312227441

Objetivo

Que el alumno conozca y utilice los principios para generar analizadores léxicos utilizando Lex.

Introducción

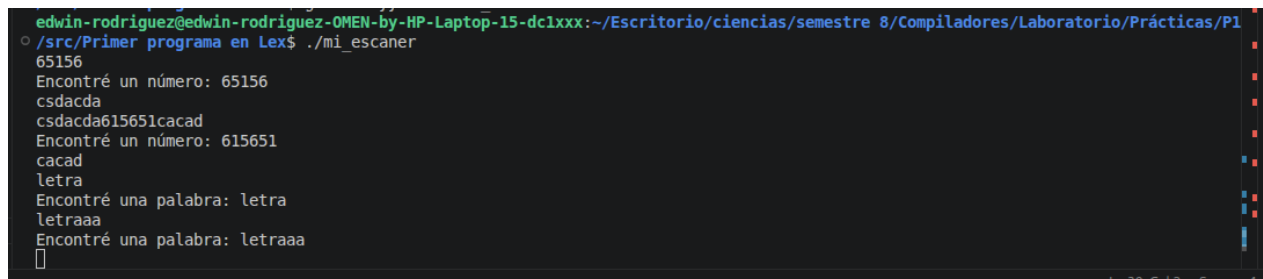
Lex es una herramienta para generar analizadores léxicos, que se deben describir mediante las expresiones regulares de los tokens que serán reconocidas por el analizador léxico (scanner o lexer). Originalmente fue desarrollado para el sistema operativo Unix, pero con la popularidad de Linux se creó una versión para este sistema llamada Flex.

Desarrollo

Primer programa en Flex++

1. ¿Qué ocurre si en la primera sección se quitan las llaves al nombre de la macro letra?

La macro `palabra` está definida usando a la macro anterior como `{letra}+`. Si quitamos las llaves y dejamos `letra+`, Flex ya no lo interpretará como la expansión de la macro que definimos arriba, sino que buscará la cadena de texto literal “letr” seguida de una o más letras “a” (la expresión regular literal).



```
edwin-rodriquez@edwin-rodriquez-OMEN-by-HP-Laptop-15-dc1xxx:~/Escritorio/ciencias/semestre 8/Compiladores/Laboratorio/Prácticas/P1
o /src/Primer programa en Lex$ ./mi_escaner
65156
Encontré un número: 65156
csdacda
csdacda615651cacad
Encontré un número: 615651
cacad
letra
Encontré una palabra: letra
letraaa
Encontré una palabra: letraaa
□
```

Figura 1: Comportamiento al quitar las llaves en la primera sección.

2. ¿Qué ocurre si en la segunda sección se quitan las llaves a las macros?

Si en la sección de reglas ponemos `palabra` en lugar de `{palabra}`, el analizador léxico buscará la cadena de caracteres exacta “palabra” en la entrada del usuario, ignorando por completo la expresión regular que le asignamos a la macro.



```
edwin-rodriguez@edwin-rodriguez-OMEN-by-HP-Laptop-15-dclxxx:~/Escritorio/ciencias/semestre 8/Compiladores/Laboratorio/Prácticas/P11
o /src/Primer programa en Lex$ ./mi_escaner
svsdv
svsdv
651561
651561
espacio

digito+
Encontré un número: digito
+
palabra
Encontré una palabra: palabra

palabraaa
Encontré una palabra: palabra
aa
```

Figura 2: Comportamiento al quitar las llaves en la segunda sección.

3. ¿Cómo se escribe un comentario en flex?

Depende de la sección del archivo. En la primer sección se puede usar `/* comentario */` sin sangrado, o `// comentario` pero debe ir con un espacio o tabulador al inicio de la línea (sangrados). En la sección de reglas (la segunda), los comentarios de las dos formas deben ir con un espacio o tabulador al inicio de la línea (sangrados), o bien estar dentro de las llaves de la acción léxica `{ /* comentario */ }`. Si se ponen al inicio del margen sin sangría, Flex marcará un error de sintaxis porque intentará leerlo como una regla. En la tercera, se usan los comentarios normales de C++ (`// comentario` o `/* comentario */`).

A continuación se muestra un ejemplo de cómo se aplican en las distintas secciones del código:

```
1 %{
2 #include <iostream>
3 %}
4
5 /* comentario en la seccion de declaraciones */
6 // comentario con sangria
7
8 %option c++
9 %option noyywrap
10
11 %%
12 /* comentario en la seccion de reglas (siempre con sangria) */
13 // otro comentario sangrado
14
15 {espacio} { /* comentario dentro de una accion lexica */ }
16
17 %%
```

```

18
19 /* comentario en la seccion de codigo de usuario (sin necesidad de
    sangria) */
20 // comentario normal de C++
21 int main() {
22     FlexLexer* lexer = new yyFlexLexer;
23     lexer->yylex();
24 }

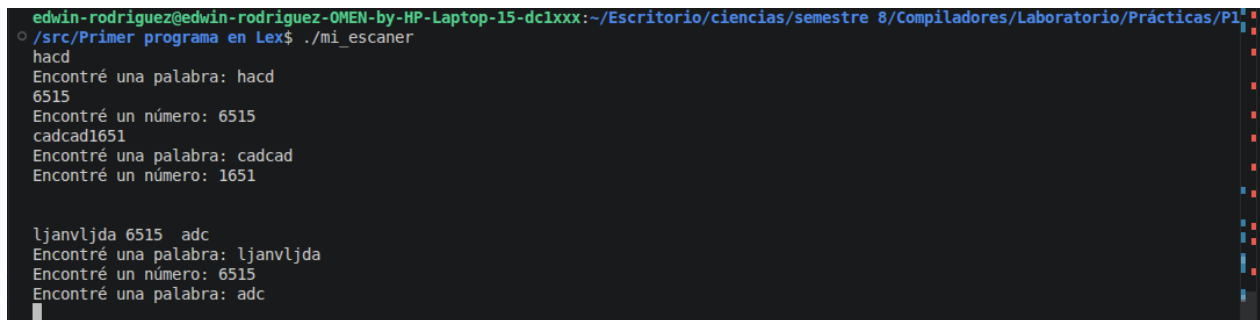
```

4. ¿Qué se guarda en yytext?

Guarda la cadena de caracteres exacta (el lexema) que acaba de hacer *match* con la regla actual. Básicamente es el pedazo de texto de la entrada que el escáner acaba de reconocer.

5. ¿Qué pasa al ejecutar el programa e introducir cadenas de caracteres y de dígitos por la consola?

El escáner lee la entrada y, gracias a las reglas, cuando detecta una secuencia de números imprime “Encontré un número: [número]”. Si detecta cadenas de caracteres, imprime “Encontré una palabra: [palabra]”. Como la regla de los espacios y saltos de línea tiene una acción vacía, el programa los ignora.



```

edwin-rodriuez@edwin-rodriuez-OMEN-by-HP-Laptop-15-dclxxx:~/Escritorio/ciencias/semestre 8/Compiladores/Laboratorio/Prácticas/P1
o /src/Primer programa en Lex$ ./mi_escaner
hacd
Encontré una palabra: hacd
6515
Encontré un número: 6515
cadcad1651
Encontré una palabra: cadcad
Encontré un número: 1651

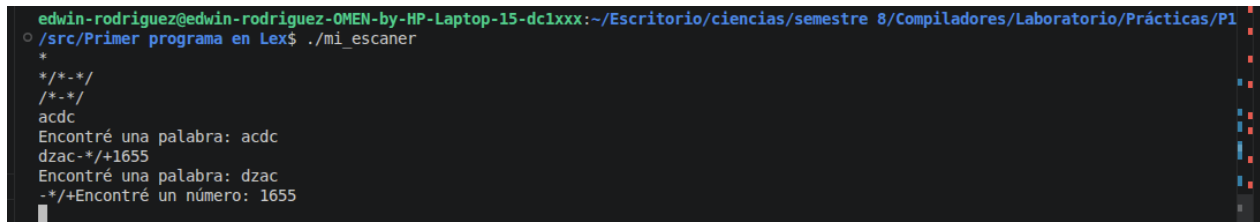
ljanvljda 6515 adc
Encontré una palabra: ljanvljda
Encontré un número: 6515
Encontré una palabra: adc

```

Figura 3: Reconociendo dígitos y cadenas.

6. ¿Qué ocurre si introducimos caracteres como “*” en la consola?

Como no escribimos ninguna regla específica para reconocer el símbolo “*”, Flex aplica su regla por defecto (*default rule*). Esta regla imprime en la pantalla cualquier carácter que no pueda emparejar con las reglas definidas.



```

edwin-rodriuez@edwin-rodriuez-OMEN-by-HP-Laptop-15-dclxxx:~/Escritorio/ciencias/semestre 8/Compiladores/Laboratorio/Prácticas/P1
o /src/Primer programa en Lex$ ./mi_escaner
*
*/+*/
/*-*/
acdc
Encontré una palabra: acdc
dzac-*/+1655
Encontré una palabra: dzac
-*/+Encontré un número: 1655

```

Figura 4: Comportamiento de la regla por defecto al introducir otros caracteres.

7. Modificar al código anterior en un archivo nuevo, de tal manera que reconozca lo siguiente:

- a) La expresión regular para los hexadecimales en lenguaje C++.
- b) 5 palabras reservadas del lenguaje C++.
- c) Los identificadores válidos del lenguaje C++, con longitud máxima de 32 caracteres.
- d) Los espacios en blanco.

A continuación se presenta el código modificado ejercicio7.11:

```
1  %{
2      #include <iostream>
3  %}
4
5  %option c++
6  %option noyywrap
7
8  /* Macros solicitadas */
9  espacio          [ ]+
10 hexadecimal      0[xX][0-9a-fA-F]+
11 identificador    [a-zA-Z_][a-zA-Z0-9_]{0,31}
12
13 %%
14
15 {espacio} {
16     std::cout << "Espacio" << std::endl;
17 }
18
19 "int"|"float"|"if"|"else"|"while" {
20     std::cout << "Palabra reservada: " << yytext << std::endl;
21 }
22
23 {hexadecimal} {
24     std::cout << "Hexadecimal: " << yytext << std::endl;
25 }
26
27 {identificador} {
28     std::cout << "Identificador: " << yytext << std::endl;
29 }
30
31 . {
32     /* Regla por defecto para caracteres no reconocidos */
33     std::cout << "Caracter no reconocido: " << yytext << std::endl;
34 }
35
36 %%
37
38 int main() {
39     FlexLexer* lexer = new yyFlexLexer;
40     lexer->yylex();
41     return 0;
42 }
```

```
edwin-rodriguez@edwin-rodriguez-OMEN-by-HP-Laptop-15-dclxxx:~/Escritorio/ciencias/semestre 8/Compiladores/Laboratorio/Prácticas/P1
/src/ejercicio7$ ./ejer7
Identificador: hola
Espacio
Identificador: com

c clksajd 651 0x651 int
Espacio
Identificador: c
Espacio
Identificador: clksajd
Espacio
Caracter no reconocido: 6
Caracter no reconocido: 5
Caracter no reconocido: 1
Espacio
Hexadecimal: 0x651
Espacio
Palabra reservada: int
```

Figura 5: Ejecución en consola.

Conceptos de Compilación en C++

Para hacer el archivo `Makefile`, es necesario conocer las cuatro fase por las que pasa el compilador GCC (g++) al procesar los archivos de código fuente (`main.cpp` y el archivo autogenerado `Lexer.cpp`):

- **Preprocesamiento:** En esta primera etapa, el compilador interpreta las directivas que comienzan con el símbolo `#`, como `#include`. Se expanden las macros del preprocesador y se incluye el contenido de los archivos de cabecera (`tokens.hpp` y `Lexer.hpp`) directamente dentro de los archivos fuente.
- **Compilación:** El código fuente de C++ ya preprocesado se traduce a código en lenguaje ensamblador, el cual es específico para la arquitectura de la CPU del sistema anfitrión. En este punto se verifica la validez de la sintaxis del lenguaje.
- **Ensamblado:** El código en ensamblador se traduce a instrucciones binarias, haciendo los archivos objeto. Estas instrucciones son legibles por el procesador pero no tienen la vinculación externa para ejecutarse de forma aislada.
- **Enlazado (Linking):** Es la fase final donde el enlazador toma los diferentes archivos objeto independientes (el resultado de `main.cpp` y el de `Lexer.cpp`) y conecta sus referencias internas con las funciones de las bibliotecas estándar del sistema (como el manejo de flujos de `<iostream>`). El resultado final es el archivo ejecutable binario denominado `compiler`.

Analizador léxico para el lenguaje C_1

8. Describir el conjunto de terminales y la expresión regular que reconoce a cada uno en *lexer.ll*.

El lenguaje C_1 tiene los siguientes terminales, definimos sus expresiones regulares:

- **Espacios en blanco:** Se utilizan para separar tokens y se ignoran.
Expresión regular: `[ \t\n\r]+`
- **Palabras reservadas:** Son `int`, `float`, `if`, `else` y `while`.
Expresión regular: `int|float|if|else|while`
- **Símbolos de puntuación y agrupación:** Incluye los paréntesis, la coma y el punto y coma.
Expresiones regulares: `\( , \) , , ;`
- **Identificadores:** Cadenas que inician con una letra o un guion bajo, seguidas opcionalmente de letras, dígitos o guiones bajos.
Expresión regular: `[a-zA-Z_][a-zA-Z0-9_]*`
- **Números:** Incluye tanto números enteros como números de punto flotante que pueden usar notación científica (por ejemplo, `1.2e6`).
Expresión regular: `[0-9]+(\.[0-9]+)?([eE][+-]?[0-9]+)?`

9. Generar acciones léxicas para cada terminal de nuestro lenguaje en *Lexer.cpp*, de modo que se muestre en pantalla la salida esperada con el archivo *prueba*.

Se incluyeron las reglas para cada token, utilizando las constantes definidas en `tokens.hpp`.

A continuación se muestra el código de `lexer.ll`:

```

1  /* =====
2  * Archivo: lexer.ll
3  * Autor: Edwin Rodriguez Nevarez
4  * Practica 1: Analizadores lexicos con Lex (Flex)
5  * Descripcion: Definicion del analizador lexico para el
6  * lenguaje C_1. Identifica palabras reservadas,
7  * identificadores, numeros y simbolos basicos.
8  * ===== */
9
10 %{\
11 #include <iostream>
12 #include <string>
13 using namespace std;
14
15 // Importamos la definicion de los tokens (ID's) y la clase Lexer
16 #include "tokens.hpp"
17 #include "Lexer.hpp"
18 %}\
19
20 %option c++
21 %option outfile="Lexer.cpp"
22 %option yyclass="C_1::Lexer"
23 %option case-insensitive
24
25 /* --- SECCION DE MACROS --- */
26 /* Ignora espacios, tabuladores, retornos de carro y saltos de linea
27    */
28 ESPACIO [ \t\n\r]+

```

```

29 /* Reconoce enteros y flotantes (con o sin notacion cientifica) */
30 NUMERO  [0-9]+(\.[0-9]+)?([eE][+-]?[0-9]+)?
31
32 /* Reconoce identificadores (letras y guiones bajos seguidos de
   alfanumericos) */
33 ID      [a-zA-Z_][a-zA-Z0-9_]*
34
35 %%
36 /* --- SECCION DE REGLAS Y ACCIONES LEXICAS --- */
37
38 {ESPACIO} { /* Accion vacia: Se ignoran los espacios en blanco */ }
39
40 /* Palabras reservadas del lenguaje C_1 */
41 "int"     { cout << C_1::INT << ", " << yytext << endl; }
42 "float"   { cout << C_1::FLOAT << ", " << yytext << endl; }
43 "if"      { cout << C_1::IF << ", " << yytext << endl; }
44 "else"    { cout << C_1::ELSE << ", " << yytext << endl; }
45 "while"   { cout << C_1::WHILE << ", " << yytext << endl; }
46
47 /* Operadores aritmeticos y de asignacion */
48 "+"       { cout << C_1::MAS << ", " << yytext << endl; }
49 "-"       { cout << C_1::MENOS << ", " << yytext << endl; }
50 "*"       { cout << C_1::MUL << ", " << yytext << endl; }
51 "/"       { cout << C_1::DIV << ", " << yytext << endl; }
52 "="       { cout << C_1::ASIG << ", " << yytext << endl; }
53
54 /* Simbolos de agrupacion y puntuacion */
55 "("       { cout << C_1::LPAR << ", " << yytext << endl; }
56 ")"       { cout << C_1::RPAR << ", " << yytext << endl; }
57 ","       { cout << C_1::COMA << ", " << yytext << endl; }
58 ";"       { cout << C_1::PYC << ", " << yytext << endl; }
59
60 /* Identificadores y Numeros */
61 {ID}      { cout << C_1::ID << ", " << yytext << endl; }
62 {NUMERO}  { cout << C_1::NUMERO << ", " << yytext << endl; }
63
64 /* Regla por defecto para atrapar errores lexicos */
65 .         { cout << "ERROR LEXICO" << yytext << endl; }
66
67 %%
68 /* --- SECCION DE CODIGO DE USUARIO --- */
69
70 /* Funcion necesaria para que Flex sepa cuando terminar de leer */
71 int yyFlexLexer::yywrap(){
72     return 1;
73 }

```

```
• edwin-rodriguez@edwin-rodriguez-OMEN-by-HP-Laptop-15-dclxxx:~/Escritorio/ciencias/semestre 8/Compiladores/Laboratorio/Prácticas/P1/src/C_1$ ./compiler
prueba
12, int
15, float
11, if
14, else
13, while
12, int
16, 12345
16, 1.2e6
10, a1
10, a 23
10,
10, id2
10, if3
10, while4
10, _b
9, ;
8, ,
6, (
7, )
12, int
7, )
10, a
10, _qbc
```

Figura 6: Salida del analizador léxico con el archivo *prueba*.

10. Crear un *Makefile*.

Este archivo primero ejecuta Flex para generar `Lexer.cpp` a partir de `lexer.ll`, y después utiliza el compilador de C++ para generar el ejecutable final llamado `compiler`. También incluye una regla `clean` para eliminar los archivos generados.

A continuación se muestra el contenido del *Makefile*:

```
1 # Variables
2 CXX = g++
3 FLEX = flex++
4 TARGET = compiler
5
6 # Regla principal
7 all: $(TARGET)
8
9 # Compilacion del programa final
10 $(TARGET): Lexer.cpp main.cpp
11     $(CXX) Lexer.cpp main.cpp -o $(TARGET)
12
13 # Generacion del escaner con Flex
14 Lexer.cpp: lexer.ll
15     $(FLEX) lexer.ll
16
17 # Limpiar archivos generados
18 clean:
19     rm -f $(TARGET) Lexer.cpp
```

```
• edwin-rodriguez@edwin-rodriguez-OMEN-by-HP-Laptop-15-dclxxx:~/Escritorio/ciencias/semestre 8/Compiladores/Laboratorio/Prácticas/P1/src/C_1$ make
flex++ lexer.ll
g++ Lexer.cpp main.cpp -o compiler
```

Figura 7: Ejecución del comando `make` en la terminal.

Extras

11. Documentar el código.

Se documentó el archivo `lexer.ll`

12. Proponer 4 archivos de prueba nuevos, 2 válidos y 2 inválidos.

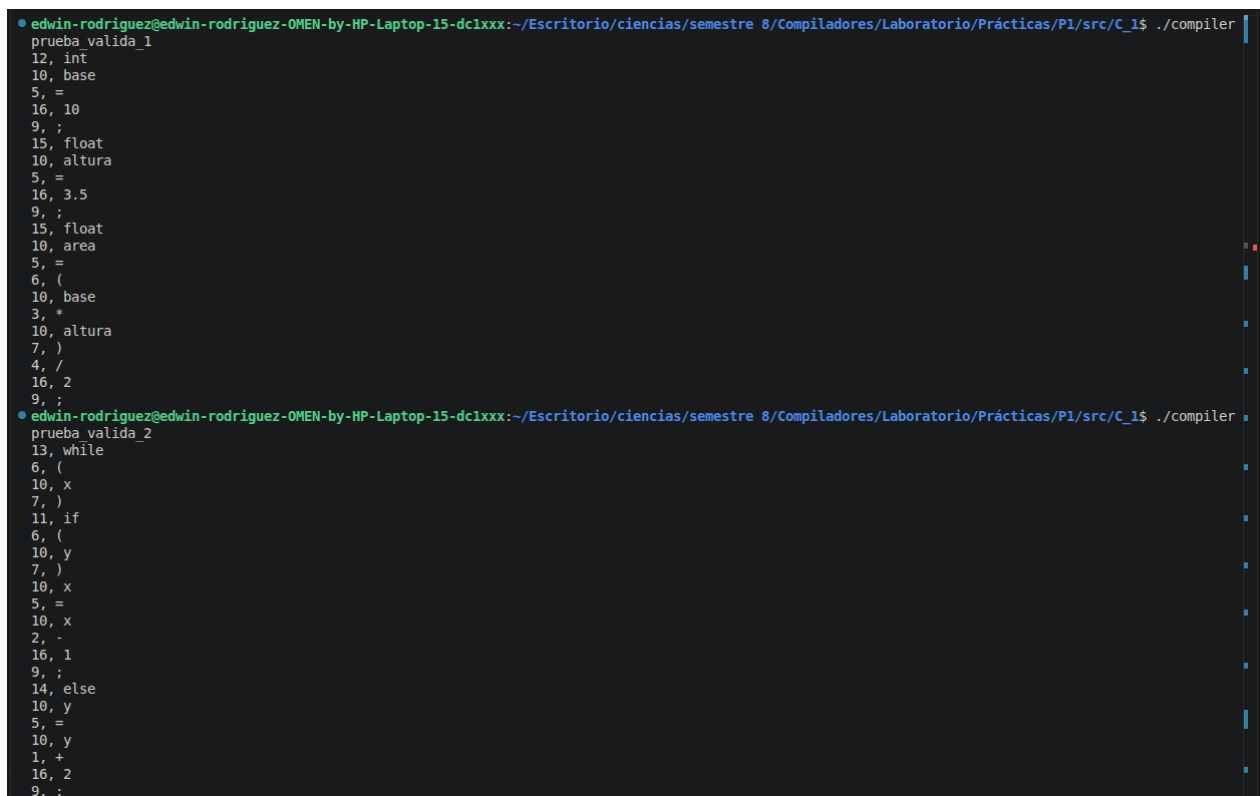
Archivos válidos:

■ prueba_valida_1:

```
1 int base = 10 ;
2 float altura = 3.5 ;
3 float area = ( base * altura ) / 2 ;
```

■ prueba_valida_2:

```
1 while ( x )
2     if ( y )
3         x = x - 1 ;
4     else
5         y = y + 2 ;
```



```
● edwin-rodriguez@edwin-rodriguez-OMEN-by-HP-Laptop-15-dclxxx:~/Escritorio/ciencias/semestre 8/Compiladores/Laboratorio/Prácticas/P1/src/C_1$ ./compiler
prueba valida_1
12, int
10, base
5, =
16, 10
9, ;
15, float
10, altura
5, =
16, 3.5
9, ;
15, float
10, area
5, =
6, (
10, base
3, *
10, altura
7, )
4, /
16, 2
9, ;
● edwin-rodriguez@edwin-rodriguez-OMEN-by-HP-Laptop-15-dclxxx:~/Escritorio/ciencias/semestre 8/Compiladores/Laboratorio/Prácticas/P1/src/C_1$ ./compiler
prueba valida_2
13, while
6, (
10, x
7, )
11, if
6, (
10, y
7, )
10, x
5, =
10, x
2, -
16, 1
9, ;
14, else
10, y
5, =
10, y
1, +
16, 2
9, ;
```

Figura 8: Salida del analizador con los archivos válidos.

Archivos inválidos:

- `prueba_invalida_1`: Con símbolos no permitidos por el escáner (@ y \$).

```
1 int costo_total = precio @ 5 ;
2 float ganancia = costo_total $ ;
```

- `prueba_invalida_2`: Utiliza elementos de C++ que no existen en C_1, como comillas " para cadenas de texto y llaves {}.

```
1 if ( a ) {
2     string mensaje = "hola" ;
3 }
```

```
edwin-rodriguez@edwin-rodriguez-OMEN-by-HP-Laptop-15-dc1xxx:~/Escritorio/ciencias/semestre 8/Compiladores/Laboratorio/Prácticas/P1/src/C_1$
• ./compiler prueba_invalida_1
12, int
10, costo_total
5, =
10, precio
ERROR LEXICO@
16, 5
9, ;
15, float
10, ganancia
5, =
10, costo_total
ERROR LEXICO$
edwin-rodriguez@edwin-rodriguez-OMEN-by-HP-Laptop-15-dc1xxx:~/Escritorio/ciencias/semestre 8/Compiladores/Laboratorio/Prácticas/P1/src/C_1$
• ./compiler prueba_invalida_2
11, if
6, (
10, a
7, )
ERROR LEXICO{
10, string
10, mensaje
5, =
ERROR LEXICO"
10, hola
ERROR LEXICO"
9, ;
ERROR LEXICO}
```

Figura 9: Salida del analizador detectando los errores léxicos en los archivos inválidos.

Análisis de Resultados y Conclusiones

Esta práctica me ayudó a comprobar que usar herramientas como Flex facilita muchísimo la creación de analizadores léxicos, ahorrándonos todo el trabajo de programar matrices de transiciones y autómatas desde cero a mano.

Al observar cómo se comportó el programa con los diferentes archivos de prueba, llegué a las siguientes conclusiones:

- **El orden de las reglas es fundamental:** Como Flex evalúa las reglas de arriba hacia abajo, es súper importante poner las palabras reservadas antes que los identificadores. Si no lo hacemos así, palabras como `while` o `int` serían tratadas como variables normales, porque también cumplen con esa regla.
- **Las macros hacen el código más limpio:** Agrupar expresiones regulares en macros y llamarlas usando llaves {} nos ayuda a no equivocarnos en reglas que son muy largas o complejas (como la de los números con punto decimal). Esto hace que el código sea mucho más fácil de leer y de modificar en el futuro.

Finalmente, como un aprendizaje y una posible mejora para las siguientes prácticas de nuestro compilador de C_1, me di cuenta de que la regla para atrapar errores (el punto .) se puede mejorar bastante. En lugar de que el programa solo imprima que hubo un error léxico y ya, podríamos hacer que también nos diga la línea y columna exacta donde encontró el símbolo no válido. Esto nos va a ayudar muchísimo a la hora de depurar el código cuando pasemos a la fase del análisis sintáctico.